Selected files

14 printable files

```
DSA\01\1a\structCalender.c
DSA\01\1b\create.c
DSA\02\string.c
DSA\03\stack.c
DSA\04\Infix_postfix.c
DSA\05\a\poseval.c
DSA\05\b\hanoi.c
DSA\06\circularQueue.c
DSA\07\SLL.c
DSA\08\DLL.c
DSA\09\2adt.c
DSA\10\bst.c
DSA\11\graph.c
DSA\12\hashing.c
```

DSA\01\1a\structCalender.c

```c
// ## PROGRAM - 01

// Develop a Program in C for the following:

// a) Declare a calendar as an array of 7 elements (A dynamically Created array) to represent 7 days of a week.
// Each Element of the array is a structure having three fields.
// The first field is the name of the Day (A dynamically allocated String),
//  the second field is the date of the Day (A integer),
// the third field is the description of the activity for a particular day (A dynamically allocated String).


#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Define a structure to represent a day
typedef struct Day {
    char *name;      // Dynamically allocated string
    int  date;
    char *activity;  // Dynamically allocated string
} DAY;

int main() {
    // Dynamically create array of 7 elements
    DAY *calendar = (DAY *)malloc(7 * sizeof(DAY));

    // Initialize with sample data using dynamic strings
    char *defaultNames[7] = {
```

```
        "Monday", "Tuesday", "Wednesday",
        "Thursday", "Friday", "Saturday", "Sunday"
    };

    char *defaultActivities[7] = {
        "Work from 9 AM to 5 PM",
        "Meeting at 10 AM",
        "Gym at 6 PM",
        "Dinner with friends at 7 PM",
        "Movie night at 8 PM",
        "Weekend getaway",
        "Relax and recharge"
    };

    for (int i = 0; i < 7; i++) {
        calendar[i].name = (char *)malloc(strlen(defaultNames[i]) + 1);
        strcpy(calendar[i].name, defaultNames[i]);

        calendar[i].activity = (char *)malloc(strlen(defaultActivities[i]) + 1);
        strcpy(calendar[i].activity, defaultActivities[i]);

        calendar[i].date = i + 1; // Just assigning 1 to 7
    }

    // Display
    printf("=====================================\n");
    printf("       Calendar for the Week       \n");
    printf("=====================================\n");

    for (int i = 0; i < 7; i++) {
        printf("%-10s (Date: %2d): %s\n", calendar[i].name, calendar[i].date, calendar[i].activity);
    }

    printf("=====================================\n");

    return 0;
}

// ### Output
// =====================================
//        Calendar for the Week
// =====================================
// Monday     (Date:  1): Work from 9 AM to 5 PM
// Tuesday    (Date:  2): Meeting at 10 AM
// Wednesday  (Date:  3): Gym at 6 PM
```

```
// Thursday   (Date:  4): Dinner with friends at 7 PM
// Friday     (Date:  5): Movie night at 8 PM
// Saturday   (Date:  6): Weekend getaway
// Sunday     (Date:  7): Relax and recharge
// ====================================
```

DSA\01\1b\create.c

```c
// ## PROGRAM - 01

// Develop a Program in C for the following:

// b) Write functions create(), read() and display(); to create the calendar, to read the data
// from the keyboard and to print weeks activity details report on screen.

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define DAYS 7
#define FILENAME "calendar.txt"

// Structure to store a day's info
typedef struct Day {
    char name[10];
    int date;
    char activity[100];
} Day;


// Function to initialize day names
void initDays(Day calendar[DAYS], const char *names[]) {
    for (int i = 0; i < DAYS; i++)
        strcpy(calendar[i].name, names[i]);
}


// Function to input calendar data and save to file
void create(Day calendar[DAYS]) {
    FILE *file = fopen(FILENAME, "w");
    if (!file) return printf("Error creating file.\n"), (void)0;

    for (int i = 0; i < DAYS; i++) {
        printf("\nEnter Date & Activity for %s: ", calendar[i].name);
        scanf("%d %[^\n]", &calendar[i].date, calendar[i].activity);
        fprintf(file, "%d\n%s\n", calendar[i].date, calendar[i].activity);
    }

    fclose(file);
    printf("\nCalendar saved to 'calendar.txt'\n");
}
```

```c

// Function to read data from file
void read( Day calendar[DAYS]) {
    FILE *file = fopen(FILENAME, "r");
    if (!file) return printf("No calendar file found.\n"), (void)0;

    for (int i = 0; i < DAYS; i++) {
        fscanf(file, "%d", &calendar[i].date);
        fscanf(file, " %[^\n]", calendar[i].activity);
    }
    fclose(file);
    printf("\nCalendar loaded from 'calendar.txt'\n");
}

// Function to display the calendar
void display( Day calendar[DAYS]) {
    printf("\n------ Weekly Calendar ------\n");
    for (int i = 0; i < DAYS; i++)
        printf("%-10s | %2d | %s\n", calendar[i].name, calendar[i].date, calendar[i].activity);
    printf("----------------------------------\n");
}

int main() {
    struct Day calendar[DAYS];
    const char *dayNames[DAYS] = {"Monday","Tuesday","Wednesday","Thursday","Friday","Saturday","Sunday"};

    initDays(calendar, dayNames);

    printf("1. Create Calendar\n2. Read Calendar\n3. Display Calender\n4. Exit\n");

    while (1){
        printf("Choose: ");
        int choice; scanf("%d", &choice);
        switch (choice) {
        case 1:
            create(calendar);
            break;
        case 2:
            read(calendar);
            break;
        case 3:
            display(calendar);
            break;
        case 4:
            printf("Exit...\n");
```

```
            exit(0);
            break;
        default:
            printf("Invalid Choice\n");
            break;
        }
    }

    return 0;
}


// ### OUTPUT

// 1. Create Calendar
// 2. Read Calendar
// 3. Display Calender
// 4. Exit
// Choose: 1

// Enter Date & Activity for Monday: 1 Read
// Enter Date & Activity for Tuesday: 2 Sleep
// Enter Date & Activity for Wednesday: 3 Get up
// Enter Date & Activity for Thursday: 4 GYM
// Enter Date & Activity for Friday: 5 work
// Enter Date & Activity for Saturday: 6 Nothing
// Enter Date & Activity for Sunday: 7 Rest

// Calendar saved to 'calendar.txt'
// Choose: 2

// Calendar loaded from 'calendar.txt'
// Choose: 3

// ------ Weekly Calendar ------
// Monday    | 1 | Read
// Tuesday   | 2 | Sleep
// Wednesday | 3 | Get up
// Thursday  | 4 | GYM
// Friday    | 5 | work
// Saturday  | 6 | Nothing
// Sunday    | 7 | Rest
// -----------------------------------
// Choose: 4
// Exit...
```

DSA\02\string.c

```c
// ## PROGRAM - 02
// 2. Design, Develop and Implement a Program in C for the following operations on Strings
// a. Read a main String (STR), a Pattern String (PAT) and a Replace String (REP)
// b. Perform Pattern Matching Operation:
// Find and Replace all occurrences of PAT in STR with REP if PAT exists in STR.
// Report suitable messages in case PAT does not exist in STR.
// Support the program with functions for each of the above operations.
// Don't use Built-in functions.

#include<stdio.h>
#include<stdlib.h>
#include<string.h>

int main(){

    char inpStr[100], searchPtr[100],  replaceStr[100], ansStr[100];
    int i=0,j=0,k=0,found=0,match=0;
    int patLen = 0;

    printf("Enter the String: ");
    fgets(inpStr,sizeof(inpStr),stdin);

    printf("Enter the Pattern To search String: ");
    fgets(searchPtr,sizeof(searchPtr),stdin);

    printf("Enter the Replace String: ");
    fgets(replaceStr,sizeof(replaceStr),stdin);

    patLen = strlen(searchPtr);
    while(inpStr[i] != '\0'){

        match = 1;

        for( j=0; searchPtr[j] != '\n' && searchPtr[j] != '\0'; j++ ){
            if (searchPtr[j] != inpStr[i + j]){
                match = 0;
                break;
            }
        }

        if(match){
            found = 1;
            for(j=0; replaceStr[j] != '\n' && replaceStr[j] != '\0'; j++){
```

```
            ansStr[k++] = replaceStr[j];
        }
        i+=(patLen-1);
    }else{
        ansStr[k++] = inpStr[i++];
    }

  }

  ansStr[k] = '\0';

  if (!found) {
    printf("\nPattern not found.\n");
    printf("Original string: %s", inpStr);
    return 0;
  }

  printf("\nResult: %s", ansStr);

  return 0;
}

// ### Output

// Enter the String: Today is Thrusday
// Enter the Pattern To search String: Thrusday
// Enter the Replace String: Saturday

// Result: Today is Saturday

// ### Output 2

// Enter the String: Today is Wednesday
// Enter the Pattern To search String: pop
// Enter the Replace String: monday

// Pattern not found.
// Original string: Today is Wednesday
```

DSA\03\stack.c

```c
// ## PROGRAM - 03

// 3) Develop a menu driven Program in C for the following operations on STACK of
// Integers (Array Implementation of Stack with maximum size MAX)
// a. Push an Element on to Stack
// b. Pop an Element from Stack
// c. Demonstrate how Stack can be used to check Palindrome
// d. Demonstrate Overflow and Underflow situations on Stack
// e. Display the status of Stack
// f. Exit
// Support the program with appropriate functions for each of the above operations

#include <stdio.h>
#include <stdlib.h>
#define MAX 5

int stack[MAX], top = -1;

void push() {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    int item;
    printf("Enter element: ");
    scanf("%d", &item);
    stack[++top] = item;
}

void pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Popped: %d\n", stack[top--]);
}

void display() {
    if (top == -1) {
        printf("Stack is Empty\n");
        return;
    }
    printf("Stack elements:\n");
```

```c
    for (int i = top; i >= 0; i--)
        printf("%d\n", stack[i]);
}

void palindrome() {
    if (top == -1) {
        printf("Stack is empty\n");
        return;
    }
    for (int i = 0, j = top; i < j; i++, j--) {
        if (stack[i] != stack[j]) {
            printf("Not Palindrome\n");
            return;
        }
    }
    printf("Palindrome\n");
}

int main() {
    int choice;
    printf("\n1.Push\n2.Pop\n3.Palindrome\n4.Display\n5.Exit\n");
    while (1) {
        printf("Choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: palindrome(); break;
            case 4: display(); break;
            case 5: exit(0);
            default: printf("Invalid choice\n");
        }
    }
}

// ### Output
// 1.Push
// 2.Pop
// 3.Palindrome
// 4.Display
// 5.Exit
// Choice: 1
// Enter element: 1
// Choice: 1
// Enter element: 2
```

```
// Choice: 1
// Enter element: 1
// Choice: 3
// Palindrome
// Choice: 2
// Popped: 1
// Choice: 2
// Popped: 2
// Choice: 2
// Popped: 1
// Choice: 1
// Enter element:
// 5
// Choice: 1
// Enter element: 4
// Choice: 4
// Stack elements:
// 4
// 5
// Choice: 5
```

DSA\04\Infix_postfix.c

```c
// 4) Develop a Program in C for converting an Infix Expression to Postfix Expression.
// Program should support for both parenthesized and free parenthesized expressions with
// the operators: +, -, *, /, % (Remainder), ^ (Power) and alphanumeric operands.

// Input -> (A+B)*(C^D-E)^(F+G*H)-I
// Output -> Infix Expression: (A+B)*(C^D-E)^(F+G*H)-I
// Postfix Expression: AB+CD^E-FGH*+^*I-

#include <stdio.h>
#include <ctype.h>
#include <math.h>
#define SIZE 50

// Stack implementation
char stack[SIZE];
int top = -1;

void push(char elem) {
    if (top < SIZE - 1)
        stack[++top] = elem;
    else
        printf("Stack Overflow!\n");
}

char pop() {
    if (top >= 0)
        return stack[top--];
    else
        return '#';
}

int precedence(char op) {
    switch (op) {
        case '(': return 1;
        case '+': case '-': return 2;
        case '*': case '/': case '%': return 3;
        case '^': return 4;
        default: return 0;
    }
}

void infixToPostfix(char infix[], char postfix[]) {
    int i = 0, k = 0;
```

```c
    char ch;
    push('#'); // Sentinel to mark bottom of stack

    while ((ch = infix[i++]) != '\0') {
        if (isalnum(ch)) {
            postfix[k++] = ch;  // Operand directly to output
        } else if (ch == '(') {
            push(ch);
        } else if (ch == ')') {
            while (stack[top] != '(')
                postfix[k++] = pop();
            pop(); // Remove '(' from stack
        } else { // Operator
            while (precedence(stack[top]) >= precedence(ch))
                postfix[k++] = pop();
            push(ch);
        }
    }

    while (stack[top] != '#')
        postfix[k++] = pop();

    postfix[k] = '\0';
}

int main() {
    char infix[SIZE], postfix[SIZE];

    printf("Enter an infix expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);

    printf("\nInfix Expression: %s", infix);
    printf("\nPostfix Expression: %s\n", postfix);

    return 0;
}


// ### Output
// Enter an infix expression: A+B*C-D
//
// Infix Expression: A+B*C-D
// Postfix Expression: ABC*+D-
```

DSA\05\a\poseval.c

```c
// ## PROGRAM - 05

// A) Develop a Program in C for the following Stack Applications a. Evaluation of Suffix expression with single digit operands and operators: +, -, *, /, %, ^

#include <stdio.h>
#include <math.h>
#define MAX 20

struct Stack {
    int top;
    float items[MAX];
} s;

void push(float x) {
    if (s.top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    s.items[++s.top] = x;
}

float pop() {
    if (s.top == -1) {
        printf("Stack Underflow\n");
        return 0;
    }
    return s.items[s.top--];
}

int isOperand(char ch) {
    return (ch >= '0' && ch <= '9');
}

float operate(float op1, float op2, char ch) {
    switch (ch) {
        case '+': return op2 + op1;
        case '-': return op2 - op1;
        case '*': return op2 * op1;
        case '/': return op2 / op1;
        case '^': return pow(op2, op1);
        default:  return 0;
    }
```

```c
}

int main() {

    s.top = -1;

    char postfix[MAX];
    printf("Enter Postfix Expression: ");
    scanf("%s", postfix);

    for (int i = 0; postfix[i] != '\0'; i++) {

        if (isOperand(postfix[i])) {
            push(postfix[i] - '0');
            printf("Pushed: %c | Stack Top: %.2f\n", postfix[i], s.items[s.top]);
        } else {
            float op1 = pop();
            float op2 = pop();
            float result = operate(op1, op2, postfix[i]);
            push(result);
            printf("Computed: %.2f %c %.2f = %.2f | Stack Top: %.2f\n",
                    op2, postfix[i], op1, result, s.items[s.top]);
        }
    }

    printf("\nFinal Answer: %.2f\n", s.items[s.top]);
    return 0;
}


// ### Output
// Enter Postfix Expression: 45+6-
// Pushed: 4 | Stack Top: 4.00
// Pushed: 5 | Stack Top: 5.00
// Computed: 4.00 + 5.00 = 9.00 | Stack Top: 9.00
// Pushed: 6 | Stack Top: 6.00
// Computed: 9.00 - 6.00 = 3.00 | Stack Top: 3.00

// Final Answer: 3.00
```

DSA\05\b\hanoi.c

```c
// ## PROGRAM - 05

// B) Solving Tower of Hanoi problem with n disks.

#include <stdio.h>
#include <math.h>

// Recursive function for Tower of Hanoi
// (n,start,temp,end)
void towerOfHanoi(int n, char from, char aux, char to) {
    if (n == 0) return;

    // (n,start,temp,end)
    towerOfHanoi(n - 1, from, to, aux);
    printf("Move disc %d from %c to %c\n", n, from, to);
    towerOfHanoi(n - 1, aux, from, to);
}

// Utility to calculate total moves
int totalMoves(int n) {
    return (int)pow(2, n) - 1;
}

int main() {
    int discs;

    printf("Enter the number of discs: ");
    scanf("%d", &discs);

    printf("\nSteps to solve Tower of Hanoi:\n");
    towerOfHanoi(discs, 'A', 'B', 'C');

    printf("\nTotal number of moves: %d\n", totalMoves(discs));

    return 0;
}

// ### Output
// Enter the number of discs: 3

// Steps to solve Tower of Hanoi:
// Move disc 1 from A to C
// Move disc 2 from A to B
```

```
// Move disc 1 from C to B
// Move disc 3 from A to C
// Move disc 1 from B to A
// Move disc 2 from B to C
// Move disc 1 from A to C

// Total number of moves: 7
```

DSA\06\circularQueue.c

```c
// ## PROGRAM - 06

// 6) Develop a menu driven Program in C for the following operations on Circular QUEUE  of Characters (Array Implementation of Queue with maximum size MAX)
// a. Insert an Element on to Circular QUEUE  b. Delete an Element from Circular QUEUE
// c. Demonstrate Overflow and Underflow situations on Circular QUEUE
// d. Display the status of Circular QUEUE e. Exit Support the program with appropriate functions for each of the above operations.

#include <stdio.h>
#include <stdlib.h>
#define MAX 3


int CQ[MAX];
int front = -1, rear = -1;


int isFull(){ return (front == ( rear + 1 ) % MAX); }
int isEmpty(){ return (front == -1); }


void insert(){

    if(isFull()){
        printf("Queue is full\n");
        return;
    }

    if(isEmpty())
        front = rear = 0;
    else rear = (rear + 1)%MAX;

    int item;
    printf("Enter element (number): ");
    scanf("%d", &item);
    CQ[rear] = item;
    printf("Inserted: %d\n",item);

}


void deleteItem(){
```

```
    if(isEmpty()){
        printf("Queue is Empty\n");
        return;
    }

    printf("Deleted: %d\n", CQ[front]);
    (front == rear) ? ( front = rear = -1) : ( front = (front+1)%MAX );
    return;
}


void display(){
    if(isEmpty()){
        printf("Queue is Empty\n");
        return;
    }

    for(int i = front; ; i=(i+1)%MAX){
        printf("%d ",CQ[i]);
        if (i==rear) break;
    }

}


int main(){

    int choice;
    printf("\n1.Insert  2.Delete  3.Display  4.Exit\n");
    while (1) {
        printf("\nEnter choice:");
        scanf("%d", &choice);
        switch (choice) {
            case 1: insert(); break;
            case 2: deleteItem(); break;
            case 3: display(); break;
            case 4: exit(0);
            default: printf("Invalid choice!\n");
        }
    }
}

// ### Output
// 1.Insert  2.Delete  3.Display  4.Exit

```

```
// Enter choice:1
// Enter element (number): 1
// Inserted: 1

// Enter choice:1
// Enter element (number): 2
// Inserted: 2

// Enter choice:1
// Enter element (number): 3
// Inserted: 3

// Enter choice:1
// Queue is full

// Enter choice:3
// 1 2 3
// Enter choice:2
// Deleted: 1

// Enter choice:3
// 2 3
// Enter choice:1
// Enter element (number): 4
// Inserted: 4

// Enter choice:3
// 2 3 4
// Enter choice:4
```

DSA\07\SLL.c

```c
// ## PROGRAM - 07

// 7. Develop a menu driven Program in C for the following operations on Singly Linked
// List (SLL) of Student Data with the fields: USN, Name, Programme, Sem, PhNo
// a. Create a SLL of N Students Data by using front insertion.
// b. Display the status of SLL and count the number of nodes in it
// c. Perform Insertion / Deletion at End of SLL
// d. Perform Insertion / Deletion at Front of SLL(Demonstration of stack)
// e. Exit

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int usn, semester;
    char name[20], branch[20], phone[20];
} STUDENT;

typedef struct node {
    STUDENT s;
    struct node *link;
} *NODE;

NODE getnode() {
    NODE x = malloc(sizeof(struct node));
    if (!x) { printf("Out of memory\n"); exit(0); }
    return x;
}

void read_student(STUDENT *s) {
    printf("Enter USN: ");
    scanf("%d", &s->usn);

    printf("Enter Name: ");
    scanf("%s", s->name);

    printf("Enter Branch: ");
    scanf("%s", s->branch);

    printf("Enter Semester: ");
    scanf("%d", &s->semester);

```

```
    printf("Enter Phone: ");
    scanf("%s", s->phone);
}


void print_student(STUDENT s) {
    printf("USN: %d, Name: %s, Branch: %s, Sem: %d, Phone: %s\n", s.usn, s.name, s.branch,
s.semester, s.phone);
}

NODE create_node(STUDENT s) {
    NODE t = getnode();
    t->s = s;
    t->link = NULL;
    return t;
}

NODE insert_end(NODE first, STUDENT s) {
    NODE t = create_node(s);
    if (!first) return t;

    NODE cur = first;
    while (cur->link) cur = cur->link;
    cur->link = t;
    return first;
}

NODE insert_beg(NODE first, STUDENT s) {
    NODE t = create_node(s);
    t->link = first;
    return t;
}

NODE delete_beg(NODE first) {
    if (!first) { printf("List empty\n"); return NULL; }

    NODE t = first;
    print_student(t->s);
    first = first->link;
    free(t);
    return first;
}

NODE delete_end(NODE first) {
    if (!first) { printf("List empty\n"); return NULL; }
    if (!first->link) {
```

```
        print_student(first->s);
        free(first);
        return NULL;
    }

    NODE cur = first, prev = NULL;
    while (cur->link) { prev = cur; cur = cur->link; }

    print_student(cur->s);
    free(cur);
    prev->link = NULL;
    return first;
}

void display(NODE first) {
    if (!first) { printf("Linked list is empty\n"); return; }

    NODE cur = first;
    int count = 0;

    printf("\nLinked list elements: \n");
    while (cur) {
        print_student(cur->s);
        cur = cur->link;
        count++;
    }
    printf("No of students = %d\n", count);
}

int main() {
    NODE first = NULL;
    STUDENT s;
    int ch, n;
    printf("\n----- MENU -----\n"
           "1 - Create SLL\n"
           "2 - Display\n"
           "3 - Insert at end\n"
           "4 - Delete end\n"
           "5 - Insert at beg\n"
           "6 - Delete beg\n"
           "7 - Exit\n"
           "----------------\n");

    while (1) {

```

```
        printf("\nEnter choice: ");
        scanf("%d", &ch);

        switch (ch) {
            case 1:
                printf("Enter no of students: ");
                scanf("%d", &n);
                while (n--) {
                    read_student(&s);
                    first = insert_end(first, s);
                }
                break;

            case 2: display(first); break;
            case 3: read_student(&s); first = insert_end(first, s); break;
            case 4: first = delete_end(first); break;
            case 5: read_student(&s); first = insert_beg(first, s); break;
            case 6: first = delete_beg(first); break;
            case 7: exit(0);
        }
    }
}


// ### Output
// ----- MENU -----
// 1 - Create SLL
// 2 - Display
// 3 - Insert at end
// 4 - Delete end
// 5 - Insert at beg
// 6 - Delete beg
// 7 - Exit
// ----------------

// Enter choice: 1
// Enter no of students: 2
// Enter USN: 1
// Enter Name: abc
// Enter Branch: ise
// Enter Semester: 3
// Enter Phone: 9620
// Enter USN: 2
// Enter Name: def
// Enter Branch: ise
```

```
// Enter Semester: 3
// Enter Phone: 974

// Enter choice: 2

// Linked list elements:
// USN: 1, Name: abc, Branch: ise, Sem: 3, Phone: 9620
// USN: 2, Name: def, Branch: ise, Sem: 3, Phone: 974
// No of students = 2

// Enter choice: 3
// Enter USN: 3
// Enter Name: ghi
// Enter Branch: ise
// Enter Semester: 3
// Enter Phone: 895


// Enter choice: 5
// Enter USN: 4
// Enter Name: klm
// Enter Branch: ise
// Enter Semester: 3
// Enter Phone: 569

// Enter choice: 2

// Linked list elements:
// USN: 4, Name: klm, Branch: ise, Sem: 3, Phone: 569
// USN: 1, Name: abc, Branch: ise, Sem: 3, Phone: 9620
// USN: 2, Name: def, Branch: ise, Sem: 3, Phone: 974
// USN: 3, Name: ghi, Branch: ise, Sem: 3, Phone: 895
// No of students = 4

// Enter choice: 4
// USN: 3, Name: ghi, Branch: ise, Sem: 3, Phone: 895

// Enter choice: 6
// USN: 4, Name: klm, Branch: ise, Sem: 3, Phone: 569

// Enter choice: 7
```

DSA\08\DLL.c

```c
// 8) Develop a menu driven Program in C for the following operations on Doubly Linked
// List (DLL) of Employee Data with the fields: SSN, Name, Dept, Designation, Sal, PhNo
// a. Create a DLL of N Employees Data by using end insertion.
// b. Display the status of DLL and count the number of nodes in it
// c. Perform Insertion and Deletion at End of DLL
// d. Perform Insertion and Deletion at Front of DLL
// e. Demonstrate how this DLL can be used as Double Ended Queue. f. Exit


#include <stdio.h>
#include <stdlib.h>
#include <string.h>


int count = 0;

typedef struct {
    char ssn[15], name[20], dept[5], desig[10];
    float sal;
    long long int ph;
} EmpData;

struct Enode {
    EmpData data;
    struct Enode *left, *right;
} *head = NULL, *tail = NULL;


struct Enode* newNode(EmpData e) {
    struct Enode *node = malloc(sizeof(struct Enode));
    node->data = e;
    node->left = node->right = NULL;
    return node;
}

void insert(EmpData e, int atEnd) {
    struct Enode *node = newNode(e);
    if (!head) {
        head = tail = node;
    } else if (atEnd) {
        tail->right = node;
        node->left = tail;
        tail = node;
```

```
    } else {
        node->right = head;
        head->left = node;
        head = node;
    }
    count++;
}

void delete(int fromEnd) {
    if (!head) {
        printf("\nList is empty!\n");
        return;
    }
    struct Enode *temp = fromEnd ? tail : head;
    printf("\nDeleted Employee (%s):\n%s %s %s %s %.2f %lld\n",
            fromEnd ? "From End" : "From Beginning",
            temp->data.ssn, temp->data.name, temp->data.dept,
            temp->data.desig, temp->data.sal, temp->data.ph);

    if (head == tail) {
        head = tail = NULL;
    } else if (fromEnd) {
        tail = tail->left;
        tail->right = NULL;
    } else {
        head = head->right;
        head->left = NULL;
    }
    free(temp);
    count--;
}

void display() {
    if (!head) {
        printf("\nList is empty.\n");
        return;
    }
    printf("\n----------------------------------------\n Linked List Elements\n----------------------------------------\n");
    for (struct Enode *temp = head; temp; temp = temp->right) {
        printf("\nSSN: %s\nName: %s\nDept: %s\nDesignation: %s\nSalary: %.2f\nPhone: %lld\n",
                temp->data.ssn, temp->data.name, temp->data.dept,
                temp->data.desig, temp->data.sal, temp->data.ph);
    }
    printf("----------------------------------------\nTotal Employees = %d\n----------------------------------------\n", count);
```

```
}

EmpData readInput() {
    EmpData e;
    printf("\nEnter ssn, name, dept, designation, salary, phone:\n");
    scanf(" %s %s %s %s %f %lld", e.ssn, e.name, e.dept, e.desig, &e.sal, &e.ph);
    return e;
}

int main() {
    int choice, n;
    EmpData e;

    printf("\n----------------- MENU --------------------\n"
           "1. Create\n2. Display\n3. Insert at Beginning\n4. Insert at End\n"
           "5. Delete at Beginning\n6. Delete at End\n7. Exit\n"
           "-------------------------------------------\n");

    while (1) {
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter number of employees: ");
                scanf("%d", &n);
                while (n--) {
                    e = readInput();
                    insert(e, 1);
                }
                break;
            case 2:
                display();
                break;
            case 3:
                e = readInput();
                insert(e, 0);
                break;
            case 4:
                e = readInput();
                insert(e, 1);
                break;
            case 5:
                delete(0);
                break;
```

```
            case 6:
                delete(1);
                break;
            case 7:
                printf("\nExit\n");
                return 0;
            default:
                printf("\nInvalid choice!\n");
        }
    }
}

// ### Output
// Same as last program
```

DSA\09\2adt.c

```c
// ## PROGRAM - 09

// 9) Develop a Program in C for the following operations on Singly Circular Linked List
// (SCLL) with header nodes a. Represent and Evaluate a Polynomial P(x,y,z) = 6x 2 y 2 z-
// 4yz 5 +3x 3 yz+2xy 5 z-2xyz 3 b. Find the sum of two polynomials POLY1(x,y,z) and
// POLY2(x,y,z) and store the result in POLYSUM(x,y,z) Support the program with
// appropriate functions for each of the above operations

// Implemented only for one variable x

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

struct node {
    int coeff, expo;
    struct node *next;
};

struct poly {
    struct node *head, *tail, *dummy;
};

/* ---------- Utility Functions ---------- */

void init_poly(struct poly *p) {
    p->dummy = (struct node *)malloc(sizeof(struct node));
    p->dummy->coeff = p->dummy->expo = 0;
    p->dummy->next = p->dummy;
    p->head = p->tail = NULL;
}

void add_term(struct poly *p, int c, int e) {
    struct node *t = (struct node *)malloc(sizeof(struct node));
    t->coeff = c;
    t->expo = e;
    t->next = p->dummy;

    if (!p->head) {
        p->head = p->tail = t;
        p->dummy->next = t;
    } else {
        p->tail->next = t;
```

```
        p->tail = t;
    }
}

void create_poly(struct poly *p) {
    int n, c, e;
    scanf("%d", &n);
    printf("Enter coef & expo:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &c, &e);
        add_term(p, c, e);
    }
}

void display_poly(struct poly *p) {
    if (!p->head) {
        printf("0\n");
        return;
    }

    struct node *t = p->head;
    int first = 1;

    while (t != p->dummy) {
        if (!first)
            printf(t->coeff >= 0 ? " + " : " - ");
        else if (t->coeff < 0)
            printf("-");

        printf("%dx^%d", abs(t->coeff), t->expo);
        first = 0;
        t = t->next;
    }
    printf("\n");
}

void add_poly(struct poly *a, struct poly *b, struct poly *c) {
    init_poly(c);
    struct node *p = a->head, *q = b->head;

    while (p != a->dummy && q != b->dummy) {
        if (p->expo == q->expo) {
            int sum = p->coeff + q->coeff;
            if (sum != 0)
                add_term(c, sum, p->expo);
```

```
            p = p->next;
            q = q->next;
        } else if (p->expo > q->expo) {
            add_term(c, p->coeff, p->expo);
            p = p->next;
        } else {
            add_term(c, q->coeff, q->expo);
            q = q->next;
        }
    }

    while (p != a->dummy) {
        add_term(c, p->coeff, p->expo);
        p = p->next;
    }
    while (q != b->dummy) {
        add_term(c, q->coeff, q->expo);
        q = q->next;
    }
}

int eval_poly(struct poly *p, int x) {
    int result = 0;
    for (struct node *t = p->head; t != p->dummy; t = t->next)
        result += t->coeff * pow(x, t->expo);
    return result;
}

/* ---------- Main ---------- */

int main() {
    int ch, x;
    struct poly p1, p2, p3;

    printf("---------------- MENU ----------------\n");
    printf("Polynomial Operations\n");
    printf("1. Add\n2. Evaluate\n3. Exit\n");
    printf("-------------------------------------\n");

    while (1) {
        printf("\nEnter your choice: ");
        scanf("%d", &ch);

        switch (ch) {

```

```
        case 1:
            init_poly(&p1);
            init_poly(&p2);

            printf("Enter no of terms of polynomial: ");
            create_poly(&p1);
            printf("The polynomial is: ");
            display_poly(&p1);

            printf("Enter no of terms of polynomial: ");
            create_poly(&p2);
            printf("The polynomial is: ");
            display_poly(&p2);

            add_poly(&p1, &p2, &p3);
            printf("Addition of polynomial is: ");
            display_poly(&p3);
            break;

        case 2:
            init_poly(&p1);

            printf("Enter no of terms of polynomial: ");
            create_poly(&p1);
            printf("The polynomial is: ");
            display_poly(&p1);

            printf("Enter value of x: ");
            scanf("%d", &x);

            printf("Value of polynomial: %d\n", eval_poly(&p1, x));
            break;

        case 3:
            printf("Exit\n");
            exit(0);
        }
    }
}


// ### Output

// ---------------- MENU ----------------
// Polynomial Operations
```

```
// 1. Add
// 2. Evaluate
// 3. Exit
// -------------------------------------

// Enter your choice: 1
// Enter no of terms of polynomial: 3
// Enter coef & expo:
// 9 6 8 5 4 2
// The polynomial is: 9x^6 + 8x^5 + 4x^2
// Enter no of terms of polynomial: 3
// Enter coef & expo:
// 10 6 5 5 3 2
// The polynomial is: 10x^6 + 5x^5 + 3x^2
// Addition of polynomial is: 19x^6 + 13x^5 + 7x^2

// Enter your choice: 2
// Enter no of terms of polynomial: 3
// Enter coef & expo:
// 10 6 5 5 3 2
// The polynomial is: 10x^6 + 5x^5 + 3x^2
// Enter value of x: 3
// Value of polynomial: 8532

// Enter your choice: 3
// Exit
```

DSA\10\bst.c

```c
// ## PROGRAM - 10

// 10) Develop a menu driven Program in C for the following operations on Binary Search
// Tree (BST) of Integers.
// a. Create a BST of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2
// b. Traverse the BST in Inorder, Preorder and Post Order
// c. Search the BST for a given element (KEY) and report the appropriatemessage
// d. Exit

#include <stdio.h>
#include <stdlib.h>

typedef struct BST {
    int data;
    struct BST *left, *right;
} NODE;

NODE* createNode(int data) {
    NODE *n = (NODE*)malloc(sizeof(NODE));
    n->data = data;
    n->left = n->right = NULL;
    return n;
}

NODE* insert(NODE *root, int data) {
    if (!root) return createNode(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);

    return root;
}

NODE* search(NODE *root, int data) {
    if (!root || root->data == data)
        return root;

    return (data < root->data) ? search(root->left, data) : search(root->right, data);
}

NODE* findMin(NODE *root) {
```

```
    while (root && root->left)
        root = root->left;
    return root;
}

NODE* deleteNode(NODE *root, int data) {
    if (!root) return NULL;

    if (data < root->data)
        root->left = deleteNode(root->left, data);
    else if (data > root->data)
        root->right = deleteNode(root->right, data);
    else {
        if (!root->left) {
            NODE *temp = root->right;
            free(root);
            return temp;
        }
        else if (!root->right) {
            NODE *temp = root->left;
            free(root);
            return temp;
        }
        NODE *temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

// Traversal

void inorder(NODE *root) {
    if (!root) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

void preorder(NODE *root) {
    if (!root) return;
    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}
```

```c

void postorder(NODE *root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}

/* -------- SEARCH WITH PARENT -------- */

void searchWithParent(NODE *root, int key) {
    NODE *parent = NULL;
    NODE *curr = root;

    while (curr != NULL) {
        if (curr->data == key) {
            printf("The %d Element is Present\n", key);
            if (parent != NULL)
                printf("Parent of node %d is %d\n", key, parent->data);
            else
                printf("Node %d is the root (no parent)\n", key);
            return;
        }

        parent = curr;
        curr = (key < curr->data) ? curr->left : curr->right;
    }

    printf("Element %d not found\n", key);
}

void menu() {
    printf("\n1.Insert");
    printf("\n2.Search");
    printf("\n3.Traversal");
    printf("\n4.Exit");
}

int main() {
    NODE *root = NULL;
    int ch, data, n;

    menu();
    while (1) {
        printf("\nEnter choice: ");
```

```
        scanf("%d", &ch);

        switch (ch) {
        case 1:
            printf("Enter number of elements: ");
            scanf("%d", &n);
            printf("Enter elements: ");
            while (n--) {
                scanf("%d", &data);
                root = insert(root, data);
            }
            break;

        case 2:
            printf("Enter element to search: ");
            scanf("%d", &data);
            searchWithParent(root, data);
            break;

        case 3:
            printf("Inorder: ");
            inorder(root);
            printf("\nPreorder: ");
            preorder(root);
            printf("\nPostorder: ");
            postorder(root);
            break;

        case 4:
            exit(0);

        default:
            printf("Invalid choice\n");
        }
    }
}


// ### Output

// 1.Insert
// 2.Search
// 3.Traversal
// 4.Exit
// Enter choice: 1
```

```
// Enter number of elements: 12
// Enter elements:
//  6 9 5 2 8 15 24 14 7 8 5 2

// Enter choice: 2
// Enter element to search: 10
// Element 10 not found

// Enter choice: 2
// Enter element to search: 15
// The 15 Element is Present
// Parent of node 15 is 9

// Enter choice: 3
// Inorder: 2 5 6 7 8 9 14 15 24
// Preorder: 6 5 2 9 8 7 15 14 24
// Postorder: 2 5 7 8 14 24 15 9 6
// Enter choice: 4
```

DSA\11\graph.c

```c
// ## PROGRAM - 11
// 11) Develop a Program in C for the following operations on Graph(G) of Cities
// a. Create a Graph of N cities using Adjacency Matrix.
// b. Print all the nodes reachable from a given starting node in a digraph using DFS/BFS methods

#include <stdio.h>
#include <stdlib.h>

#define MAX 20

int n;
int adj[MAX][MAX];
int visited[MAX];


void reset(int arr[]) {
    for (int i = 1; i <= n; i++)
        arr[i] = 0;
}


void readGraph() {
    printf("Enter graph data in matrix form:\n");
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            scanf("%d", &adj[i][j]);
}


void dfs(int v, int *count) {
    visited[v] = 1;

    for (int i = 1; i <= n; i++) {
        if (adj[v][i] && !visited[i]) {
            printf("%d->%d ", v, i);
            (*count)++;
            dfs(i, count);
        }
    }
}


void bfs(int start) {
```

```
    int queue[MAX], front = 0, rear = -1;

    visited[start] = 1;
    queue[++rear] = start;

    while (front <= rear) {
        int v = queue[front++];
        for (int i = 1; i <= n; i++) {
            if (adj[v][i] && !visited[i]) {
                visited[i] = 1;
                queue[++rear] = i;
            }
        }
    }
}


void menu() {
    printf("\n1. BFS\n2. DFS\n3. Exit\n");
}


int main() {
    int choice, start;

    printf("Enter the number of vertices:");
    scanf("%d", &n);

    readGraph();
    menu();
    scanf("%d", &choice);

    switch (choice) {

    case 1:
        printf("Enter the starting vertex:");
        scanf("%d", &start);

        if (start < 1 || start > n) {
            printf("BFS is not possible\n");
            break;
        }

        reset(visited);
        bfs(start);
```

```

        printf("The nodes which are reachable from %d: ", start);
        for (int i = 1; i <= n; i++)
            if (visited[i])
                printf("%d ", i);
        break;

    case 2: {
        int edgeCount = 0;
        reset(visited);
        dfs(1, &edgeCount);

        if (edgeCount == n - 1)
            printf("\nGraph is connected");
        else
            printf("\nGraph is not connected");
        break;
    }

    case 3:
        exit(0);

    default:
        printf("Invalid choice\n");
    }

    return 0;
}


// ### Output 1
// Enter the number of vertices:5
// Enter graph data in matrix form:
// 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 0 0 1 0 0 0

// 1. BFS
// 2. DFS
// 3. Exit
// 2
// 1->2 2->3 3->4 2->5
// Graph is connected

// ### Output 2

// Enter the number of vertices:5
```

```
// Enter graph data in matrix form:
//  0 1 0 1 0 1 0 1 0 0 0 1 0 1 0 1 0 1 0 0 0 0 0 0 0

// 1. BFS
// 2. DFS
// 3. Exit
// 2
// 1->2 2->3 3->4
// Graph is not connected

// ### Output 3

// Enter the number of vertices:5
// Enter graph data in matrix form:
// 0  1 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0

// 1. BFS
// 2. DFS
// 3. Exit
// 1
// Enter the starting vertex:1
// The nodes which are reachable from 1: 1 2 3 4

// ### Output 4

// Enter the number of vertices:5
// Enter graph data in matrix form:
// 0 1 1 0 0 0 0 0 1
// 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0

// 1. BFS
// 2. DFS
// 3. Exit
// 1
// Enter the starting vertex:0
// BFS is not possible
```

DSA\12\hashing.c

```c
// ## PROGRAM - 12

// 12) Given a File of N employee records with a set K of Keys (4-digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table (HT) of m memory locations with L as the set of memory addresses (2-digit) of locations in HT.

// Let the keys in K and addresses in L are Integers.
// Develop a Program in C that uses Hash function H: K →L as H(K)=K mod m
// (remainder method), and implement hashing technique to map a given key K to the address space L. Resolve the collision (if any) using linear probing

#include <stdio.h>
#include <stdlib.h>

#define MAX 10
#define EMPTY -1

int hash(int key) {
    return key % MAX;
}

void init(int ht[]) {
    for (int i = 0; i < MAX; i++)
        ht[i] = EMPTY;
}

void insert(int ht[], int key) {
    int index = hash(key);

    if (ht[index] == EMPTY) {
        ht[index] = key;
        return;
    }

    printf("\nCollision Detected...!!!\n");
    printf("Collision avoided successfully using LINEAR PROBING\n");

    for (int i = (index + 1) % MAX; i != index; i = (i + 1) % MAX) {
        if (ht[i] == EMPTY) {
            ht[i] = key;
            return;
        }
    }
```

```c

    printf("Hash table is full\n");
    exit(1);
}

void display(int ht[]) {
    int choice;
    printf("\n1. Display ALL\n2. Filtered Display\n");
    scanf("%d", &choice);

    printf("\n--------------------------------\n");
    printf("Index\t\tKey\n");
    printf("--------------------------------\n");

    for (int i = 0; i < MAX; i++) {
        if (choice == 1 || ht[i] != EMPTY)
            printf("%d\t\t%d\n", i, ht[i]);
    }

    printf("--------------------------------\n");
}


int main() {
    int ht[MAX], key, ch = 1;

    printf("collision handling by linear probing :\n");
    init(ht);

    while (ch) {
        printf("\nEnter the data: ");
        scanf("%4d", &key);
        insert(ht, key);
        printf("Do you wish to continue ? (1/0): ");
        scanf("%d", &ch);
    }

    display(ht);
    return 0;
}


// ### Output
// collision handling by linear probing :

```

```
// Enter the data: 20
// Do you wish to continue ? (1/0): 1

// Enter the data: 50

// Collision Detected...!!!
// Collision avoided successfully using LINEAR PROBING
// Do you wish to continue ? (1/0): 1

// Enter the data: 3
// Do you wish to continue ? (1/0): 1

// Enter the data: 6
// Do you wish to continue ? (1/0): 0

// 1. Display ALL
// 2. Filtered Display
// 2

// ---------------------------------
// Index        Key
// ---------------------------------
// 0            20
// 1            50
// 3            3
// 6            6
// ---------------------------------
```